LeeTKOB — the class project

CSC13114 · class `2609-AWAD-23/3` · from `PA#2` to `PA#6`, every team builds this product

LeeTKOB was proposed in `PA#1` by **Lê Tấn Hiệp (23120255), Tống Dương Thái Hoà (23120262) and Nguyễn Phúc Hoàng (23120264)**. From now on they are the class's **product owners** (section 8), and each of them earns a bonus of **0.5 points on the course mark** (10-point scale).

Your own `PA#1` proposal keeps its mark. From `PA#2` on, every team builds LeeTKOB.

1. Why one product for the whole class

- **Your evals become comparable.** Every team's constraint checker is scored on the same held-out set, which the lecturer keeps (section 6). That makes "zero wrongful deductions" a number we can compare, not a claim.
- **Red-teaming has a target.** In `PA#4` you attack another team's deployment of the same product. You know exactly what it is supposed to refuse.
- **You compete on engineering, not on the idea.** The idea is fixed. What gets marked is how the harness, the gates, the eval and the operations hold up, and whether you can explain them in the oral.

2. The user and the problem

The user is a teaching assistant (TA) running an introductory C++ programming lab. It is the night before the marks for the *Sorting* lab are published. About 80 students have submissions marked **Accepted**, and the TA is reading them one by one to check whether each student really implemented merge sort.

The problem: an Accepted submission can still break the assignment's rules. It may call `std::sort`, or it may implement bubble sort when merge sort was required. Today the only way to find out is to read every submission, which takes 2–3 minutes each and 3–4 hours a week. Students sometimes appeal a deduction, and the TA has to re-read the code to defend it.

3. The LLM feature and the cost of being wrong

Constraint review before marks are published. This is one step in the grading pipeline, not a chatbot.

- **Input:** the lab statement, the constraints the TA declared (each with a text and a deduction the TA chose), and the code of an Accepted submission with comments and the student's identity stripped.
- **Hard rules run first.** Syntax-aware checks catch certain violations, such as a call to `std::sort` (as opposed to the student's own function that happens to be called `sort`). When a hard rule decides a constraint, the model is not called for it.
- **The model judges only the semantic constraints.** Examples: whether the required algorithm is really the one used, whether recursion is really present, whether an auxiliary array is hidden somewhere. For each constraint it returns `pass / violation / unsure` with the code lines that prove it. **It never decides the deduction.** The TA set that.

- **Guardrails.** The output is validated against a schema. Every evidence line must match the submitted code exactly, or the verdict becomes `unsure`. A timeout also becomes `unsure`. There is a cost cap per lab.
- **Human gate.** Every `violation`, every `unsure`, and a random 10% of the `pass` verdicts go to the TA's review queue. **Marks are published only after the TA has reviewed them and pressed Publish.** That is the step that is hard to undo.

Why being wrong is expensive. A wrongful deduction (for example 3 marks out of 10, in a lab worth 30% of the course) can push a student below the pass mark or below a scholarship threshold. If one constraint is misread systematically, all 80 students lose marks at once and up to 80 appeals follow. Before publishing, fixing a verdict costs nothing. After publishing, marks have to be reopened, the appeals handled, and the TA loses credibility.

The bar. The checker must not deduct wrongly more often than a careful human does:

Metric	Threshold
Wrongful deductions on compliant solutions	**0**
Violations caught (verdict `violation` with a correct evidence line)	**≥ 90%**
`unsure` among all verdicts	**≤ 20%**

Data sent to the model: only the statement, the constraints, and code with names and student IDs removed. Students are told about the review when they submit.

4. Scope

In: what LeeTKOB does by the end of the semester

1. **Accounts and roles:** admin, TA and student. TAs belong to classes. Each lab belongs to a class.
2. **Submitting and judging C++:** a student submits, the submission goes onto a **queue**, and it is compiled and run in a **sandbox** against the TA's tests. The student sees the result in real time.
3. **Constraint review:** hard rules, then the model, as described in section 3. The TA works through a review queue, then publishes the marks.
4. **Appeals:** after publishing, a student can appeal a deduction, and the TA accepts it or rejects it with a reason.
5. **Quality:** an eval on the dev set runs in CI and **blocks the merge** when any wrongful deduction appears.
6. **Load:** the system holds up when 80 submissions arrive at once (the last minutes before a deadline), and you can show a load-test result for it.
7. **Deployment:** a public deployment that the lecturer and your red team can reach.

Out: deliberately not built

- **A second programming language.** It would need a second set of hard rules.
- **Tests generated by the model, or the model choosing the deduction.** The TA writes the tests and sets the deductions.
- **Plagiarism or AI-written-code detection.** We cannot measure it properly within a semester.

- **Contest rankings, an online IDE, vector databases / RAG.** None of these helps a TA check a lab.

5. What is your team's choice

- **Stack and architecture.** Any language, any framework, monolith or services. The original proposal used three services in three languages with five data stores. You do not have to follow it. Fewer moving parts is a legitimate engineering decision, and you should be ready to defend whichever you pick in the oral.
- **Model provider and model.** Free tiers are fine. Set a spending cap, and state in `AI-LOG.md` and in your README which model you use and what one lab costs.
- **The sandbox:** your own container setup, or a self-hosted Judge0. Either way it must meet the minimums below.

The sandbox is not optional, and its minimums are fixed

Your system runs code written by strangers. Every team's sandbox must:

- have **no network access**;
- limit **CPU time, wall time, memory, number of processes and output size**;
- run as a **non-root** user, with no host mounts and **no access to the Docker socket**;
- receive **no secrets**: the model API key must never be visible inside the sandbox, including in its environment variables.

You prove each of these with a test case in your repository: a submission that tries the attack and fails. Session 6 (Thursday, Oct 29) is about this.

6. The eval data

- **Dev set:** C++ lab problems, each with tests, constraints and Accepted solutions labelled per constraint (`pass` / `violation`, with evidence lines). It also includes disguised violations, compliant "traps" and prompt-injection attempts, plus a zero-dependency `eval.mjs` that applies the thresholds in section 3. The lecturer publishes it before session 7 (Thursday, Nov 5). Add your own cases on top of it.
- **Held-out set:** different problems in the same format, kept by the lecturer. Your `PA#4` checker is run on it. Train your prompts on the dev set, not on guesses about the held-out set.
- **Your prediction file** is JSONL, one line per (solution, constraint):

```
{"solution":"sort-merge/s03","constraint":"C1","verdict":"violation","evidence":
[{"line":12,"text":"std::sort(a, a + n);"}]}
```

Make your checker able to write this file from the command line. The lecturer runs the held-out set through exactly that command.

7. Checkpoints, for LeeTKOB

Dates and topics are the ones already on Classroom (*Project checkpoints PA#2–PA#6*). The rubric for each checkpoint is attached to its own assignment when the assignment opens.

Checkpoint	Due, 22:00	What it means for LeeTKOB
`PA#2` harness and quality gate	Wed Nov 11	Convention file, lint, tests, and CI that really blocks a merge. Plus the **contract of one LeeTKOB endpoint** (`docs/contract/<endpoint>.md`) and contract tests that run in CI.
`PA#3` prototype with the LLM feature running	Wed Nov 25	A TA creates a lab with constraints. A student submits, and the code is judged in the sandbox through the queue. The Accepted submission goes through hard rules and then the model. The verdicts, with evidence, appear in the TA's review queue.
`PA#4` eval, guardrail, human gate, red-team	Wed Dec 2	The eval on the dev set gates CI. The guardrails of section 3 are in place. Publishing is impossible without the TA's review. You also write a red-team report on another team's deployment, and the lecturer assigns the pairs.
`PA#5` final build, running and observed	Wed Dec 16	Publishing and appeals, real-time results, logs/metrics/traces, a load test with 80 simultaneous submissions, a public deployment, and one incident post-mortem.
`SEMINAR`	Dec 10, Dec 17	Unchanged: one optimisation topic per team.
`PA#6` individual oral	Dec 24, Dec 31	On **your team's** code and decisions.

8. Product owners

The authoring team decides **how the product behaves** wherever this brief is silent. For example: can a student see the evidence before marks are published? What does the TA see for an `unsure` verdict? Can an appeal be reopened?

- **Ask** in the class Facebook group, under the pinned post *LeeTKOB — questions for the product owners*. Do not ask in private messages: one team's answer is everyone's answer.
- **Answers** come within two working days. They are collected in the product owners' FAQ, linked on Classroom. Cite the FAQ entry in your repository when you rely on it.
- **What the product owners do not decide:** marks, deadlines, rubrics, and the thresholds and minimums in this brief. Those belong to the lecturer. If the FAQ and this brief disagree, this brief wins until the lecturer changes it.
- The product owners build LeeTKOB like every other team and are marked the same way.